

Impact of Image Resolution and Class Complexity on QCNN Performance

Ashmit Sharma¹ and Shitala Prasad²

¹ Shri G.S. Institute of Technology, Indore, India

² Indian Institute of Technology, Goa, India

ashmitsharma630@gmail.com & shitala@iitgoa.ac.in

Abstract. Quantum Convolutional Neural Networks (QCNNs) have recently emerged as a promising approach for enhancing various computer vision tasks, such as object detection, image segmentation, and classification. By leveraging quantum computing principles and utilizing qubits instead of classical bits, QCNNs offer potential improvements over traditional Convolutional Neural Networks (CNNs). This paper presents a simulated QCNN model based on the framework proposed by Caro et al. (2022) to analyze its performance across different classification tasks. We evaluate the QCNN's effectiveness on two distinct datasets with varying image dimensions, providing insights into its strengths and limitations. Our analysis identifies the scenarios in which QCNNs outperform classical CNNs and where their performance may be constrained, contributing to a better understanding of the impact of image size and class complexity on QCNN performance.

Keywords: Quantum Convolutional Neural Networks · Image Classification · Quantum Machine Learning · Qbits

1 Introduction

Image recognition and classification are fundamental tasks in computer vision with applications spanning numerous industries, including healthcare, autonomous driving, security, and retail. Accurate image recognition enables innovations such as early disease detection, automated navigation systems, and intelligent surveillance. To address these tasks, the field of computer vision has witnessed significant advancements, resulting in the development of a variety of machine learning models. Among these, Convolutional Neural Networks (CNNs) have consistently outperformed others, becoming the backbone of most state-of-the-art (SOTA) solutions for image classification, object detection, and image segmentation [5].

CNNs, introduced by LeCun et al. (1998), leverage a hierarchical structure composed of multiple layers to extract spatial hierarchies of patterns from input images [5]. The architecture typically includes convolutional layers, which apply a set of filters to the input data to capture local patterns like edges and textures, and pooling layers, which down-sample the data to reduce dimensionality while preserving important features. Fully connected layers then interpret

these extracted features to perform the final classification. The end-to-end training of CNNs allows them to learn complex representations, making them highly effective for a wide range of visual recognition tasks.

However, despite their powerful capabilities, CNNs have several critical limitations. First, they require large, well-annotated datasets to generalize well, which are often difficult and expensive to acquire in many real-world applications. Second, their training process is computationally intensive, demanding high processing power and substantial memory resources, especially as the size and complexity of the network increase. Additionally, CNNs are typically deterministic, relying on classical bits, which limits their ability to efficiently process vast amounts of high-dimensional data, especially as the demand for real-time applications grows.

To address these limitations, researchers have been exploring alternative approaches, leading to the emergence of Quantum Convolutional Neural Networks (QCNNs). QCNNs are an innovative extension of classical CNNs that operate in the quantum domain, leveraging the principles of quantum computing. While QCNNs maintain a similar layered architecture to that of CNNs—including quantum equivalents of convolutional, pooling, and fully connected layers—they utilize quantum bits (qubits) instead of classical bits, enabling them to take advantage of quantum phenomena such as superposition and entanglement.

Quantum Convolutional Neural Networks employ quantum variational circuits to mimic the operations of traditional CNN layers [2]. In QCNNs, the convolutional layers are replaced with quantum circuits designed to extract quantum features from the input data, while quantum pooling layers reduce the data size, similar to classical pooling operations, but with the potential for more efficient data compression due to quantum entanglement. The fully connected layers in a QCNN are also implemented using quantum circuits that map the extracted features to the output labels. This approach allows QCNNs to operate on high-dimensional data with potentially exponential speed-ups over their classical counterparts.

The use of quantum bits introduces several advantages. QCNNs can potentially process complex data with fewer resources due to quantum parallelism, enabling more efficient computation for certain types of problems. They can also explore multiple solutions simultaneously, which may enhance their ability to generalize from smaller datasets, reducing the need for extensive labeled data. Moreover, QCNNs have been implemented for various computer vision tasks such as image classification, object detection[8], and image segmentation[3]. These networks have the potential to mitigate some of the computational bottlenecks faced by classical CNNs, making them more suitable for real-time applications and large-scale data analysis.

In this study, we explore the performance of QCNNs on different image classification tasks, comparing them with traditional CNNs to assess their relative strengths and weaknesses. We focus on the impact of varying image dimensions and class sizes to understand where QCNNs may offer significant advantages and where they might face limitations. By providing a detailed analysis of these

emerging quantum models, we aim to contribute to the ongoing research on the integration of quantum computing with machine learning to solve complex computer vision problems.

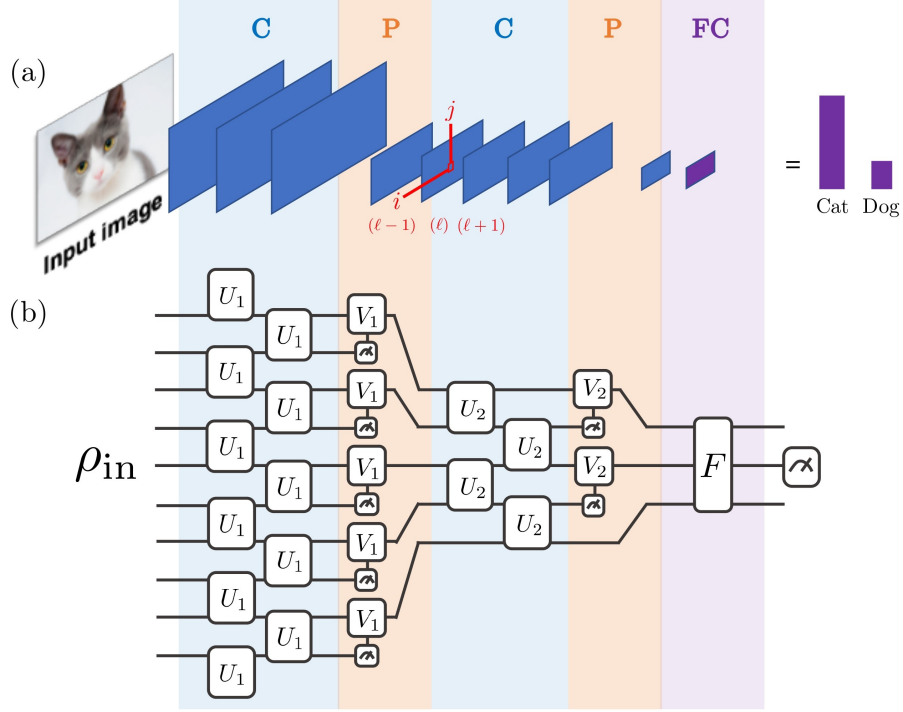


Fig. 1: (a) Overview of classical CNN showing a series of image processing layers - Convolution layer(C), pooling layer(P), and fully connected layer(FC) - that converts an input image into a set of feature maps(blue rectangles), ultimately resulting in an output probability distribution (purple bars).
(b) Overview of QCNN - Inheriting a similar layered structure[2]

The motivation for this study stems from the need to overcome the limitations of traditional CNNs in practical, real-world applications. QCNNs offer the potential to perform critical tasks such as clustering, classification, and optimization—key components in many AI systems—at speeds exponentially faster than classical methods. Additionally, QCNNs are better equipped to handle complex data, making them a promising alternative for advancing AI capabilities.

In this work, we simulate a QCNN model composed of quantum variational circuits and quantum gates using pennylane framework for image classification. We conduct a number of experiments across multiple datasets with varying dimensions including MNIST and CIFAR10. The major aim of this work is to assess the efficacy of the simulated QCNN model in comparison to a classical

CNN, identifying where the simulated QCNN excels and where does it fall short in comparison to CNN model.

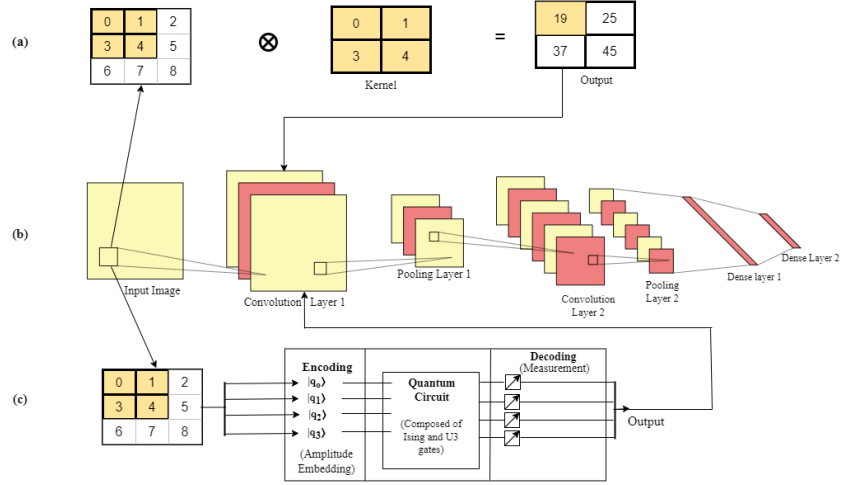


Fig. 2: (a) Convolution operation in a classical CNN (b) Basic layered architecture of a CNN and a QCNN (c) Convolution operation in a QCNN[2]

The structure of the paper is as follows: Section 2 provides a detailed description of the CNN and QCNN architectures used in this study. Section 3 presents the experiments conducted and the corresponding results, followed by an in-depth discussion in Section 4. Finally, Section 5 concludes with a summary of the experimental analysis.

2 Methodology

This section outlines the methodology used to implement the simulated QCNN model and assess its performance.

2.1 Datasets

In this study, we utilized two widely recognized datasets: the Modified National Institute of Standards and Technology (MNIST) dataset [5] and the Canadian Institute for Advanced Research 10 Classes (CIFAR-10) dataset [4]. Our objective was to benchmark the performance of our simulated QCNN model against the results achieved by Caro et al. (2022) [1], which employed the scikit-learn digits dataset [7].

MNIST The MNIST dataset is a foundational dataset in the fields of machine learning and computer vision, consisting of 70,000 grayscale images of handwritten digits ranging from 0 to 9. It includes 60,000 images for training and 10,000 for testing, with each image having a resolution of 28×28 pixels. Due to its simplicity, standardization, and the variety of techniques previously applied to it, MNIST serves as a valuable benchmark for evaluating the effectiveness of various image classification models.

CIFAR10 The CIFAR-10 dataset, on the other hand, presents a more challenging and diverse set of images. It contains 60,000 color images categorized into 10 distinct classes, such as airplanes, automobiles, and animals, providing a broader spectrum of complexity compared to MNIST. Each class comprises 6,000 images, with the dataset divided into 50,000 training samples and 10,000 testing samples, each with a resolution of 32×32 pixels. The complexity and variability within CIFAR-10 make it an ideal choice for assessing a model's versatility and generalization capabilities across different types of images.

To ensure a comprehensive comparison, we also considered the dataset used by Caro et al., which is the scikit-learn digits dataset. This dataset consists of 1,797 grayscale images of handwritten digits from 0 to 9, with each image sized at 8×8 pixels. While more limited in scale and resolution than MNIST and CIFAR-10, this dataset provides another perspective for evaluating the performance of QCNN models on simpler image classification tasks.

By leveraging these three datasets, we aim to assess the simulated QCNN model's performance across a wide range of image classification scenarios, from simpler tasks involving smaller images to more complex tasks with diverse and higher-resolution datasets.

2.2 Pre-processing

To enable a direct and comprehensive comparison with Caro et al.'s model [1], which uses images of size 8×8 pixels, we applied several pre-processing steps to the MNIST and CIFAR-10 datasets.

1. Downsizing: The original MNIST images of size 28×28 pixels and CIFAR-10 images of size 32×32 pixels were resized to 8×8 pixels using average pooling. Let I represent the original image, and let I' denote the downsized image. Average pooling was applied over non-overlapping regions of size $r \times r$, where $r = \frac{\text{original_size}}{8}$. The value of each pixel p'_{ij} in the downsized image I' is calculated as:

$$p'_{ij} = \frac{1}{r^2} \sum_{m=r_i}^{r_i+r-1} \sum_{n=r_j}^{r_j+r-1} p_{mn} \quad (1)$$

where p_{mn} represents the pixel values in the original image I . This reduces the spatial resolution while preserving essential information.

2. Grayscale: Additionally, the CIFAR-10 dataset, originally composed of color images with three channels (RGB), was converted to grayscale to match the single-channel format of the MNIST and scikit-learn digits datasets. For an image pixel (R, G, B) , the grayscale intensity Y was computed using the formula:

$$Y = 0.2989R + 0.5870G + 0.1140B \quad (2)$$

where R, G , and B represent the red, green, and blue components, respectively. This conversion simplifies the dataset by reducing it to a single channel while maintaining the overall brightness and contrast characteristics.

By applying these pre-processing steps, we ensure that all datasets are comparable in terms of resolution and color format, enabling a more accurate assessment of the model's performance across different data conditions.

2.3 Architecture

The architecture of Quantum Convolutional Neural Networks (QCNNs) is designed to mirror the structure of classical Convolutional Neural Networks (CNNs), with a fundamental difference in how the data is processed. In QCNNs, conventional layers such as convolutional and pooling layers are replaced by quantum circuits, leveraging quantum principles like entanglement and superposition. These quantum circuits consist of quantum gates that manipulate information in ways that classical circuits cannot, enabling the capture of more complex data correlations [9].

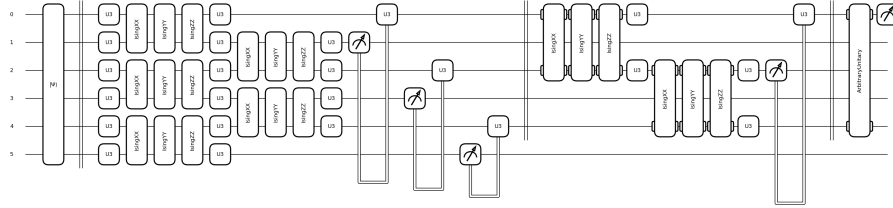


Fig. 3: QCNN architecture - Using 6 qubits for encoding of images having a dimension of 8×8 pixels and 1 qubit for measuring the probabilities of binary classification

Encoding The QCNN begins with an input state, denoted as $|\psi\rangle$, which represents the encoded input data. In this model, amplitude embedding is employed to map classical data into a quantum state. This is achieved by setting the amplitude of each computational basis state according to the input values, allowing for an efficient and compact representation of classical data in a quantum framework.

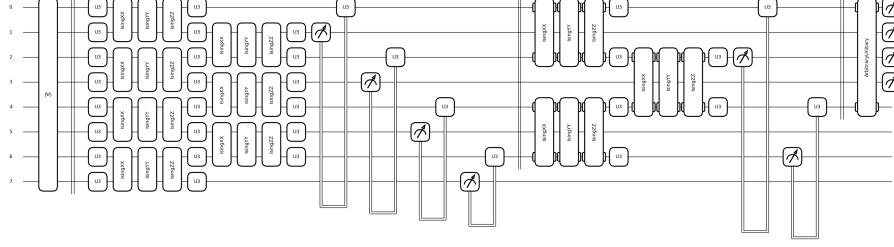


Fig. 4: QCNN architecture - Using 8 qubits for encoding of images having a dimension of $8 \times 8 \times 3$ pixels and 4 qubits for measuring the probabilities of classification into 10 classes

U3 gates The U3 gate[6] is a universal single-qubit quantum gate that can perform any arbitrary rotation on a qubit. It is defined by three parameters θ , ϕ , and λ and is represented as follows:

$$U3(\theta, \phi, \lambda) = \begin{pmatrix} \cos(\theta/2) & -e^{i\lambda} \sin(\theta/2) \\ e^{i\phi} \sin(\theta/2) & e^{i(\phi+\lambda)} \cos(\theta/2) \end{pmatrix}$$

This gate can perform a wide range of operations and is a generalization of other single-qubit gates like the Pauli gates and the Hadamard gate.

Ising Gates Ising gates are multi-qubit gates used in quantum computing, primarily based on the Ising model[10] from statistical mechanics. They are crucial for implementing entangling operations between qubits. A common form of Ising gate is the ZZ gate, which can be described as follows:

$$ZZ(\theta) = \exp(-i\theta Z \otimes Z/2)$$

Where Z is the Pauli-Z matrix. The Ising gate applies an interaction between pairs of qubits and is used to simulate the evolution of a quantum system described by the Ising model.

Convolutional Layer In the quantum convolutional layer, a series of quantum gates is applied to the qubits for feature extraction. This layer functions as a set of two-qubit unitary transformations that process the input data by correlating adjacent qubits through a sliding kernel across the entire image.

To implement the convolutional layer, we define the weights of the two-qubit unitary transformation, which are adjusted during each training iteration. In the PennyLane framework, this two-qubit unitary operation is constructed using a specific sequence of quantum gates. The sequence begins with two single-qubit unitary gates, each parameterized by three adjustable angles. These are followed by three Ising interaction gates between the pair of qubits, each governed by a

single tunable parameter. Finally, the layer concludes with an additional set of U3 gates applied to each qubit, completing the feature extraction process.

The sequence in each layer is as follows:

1. $U3(w_1, w_2, w_3)$ and $U3(w_4, w_5, w_6)$ on the first pair of qubits.
2. $\text{IsingXX}(w_7)$, $\text{IsingYY}(w_8)$, $\text{IsingZZ}(w_9)$.
3. $U3(w_{10}, w_{11}, w_{12})$ and $U3(w_{13}, w_{14}, w_{15})$ and so on for the rest of the qubits.

Here W_i are the weights which acts as parameters for the Ising and U3 gates.

Pooling Layer In the pooling layer of our QCNN, we utilize a conditional single-qubit unitary operation that depends on the measurement outcomes of neighboring qubits. The pooling layer processes input data by applying single-qubit unitaries, specifically U3 gates, which are parameterized and adjusted during training. After applying these unitaries, conditional measurements are performed on a subset of qubits, effectively reducing the size of the quantum system by a factor of 2. This reduction simplifies the quantum state while preserving essential information, akin to pooling operations in classical neural networks that downsample feature maps to reduce dimensionality.

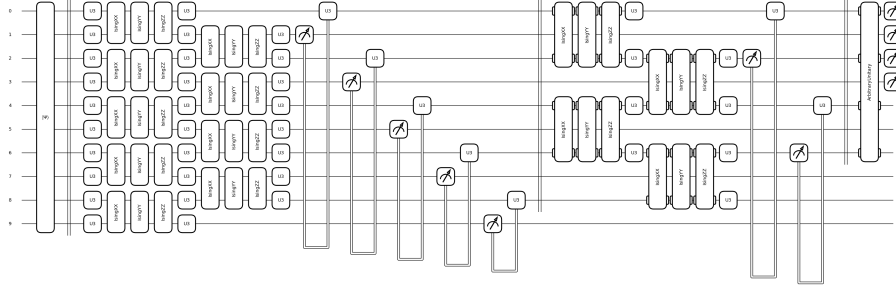


Fig. 5: QCNN architecture - Using 10 qubits for encoding of images having a dimension of 28x28 pixels and 4 qubits for measuring the probabilities of classification into 10 classes

QCNN Circuit Construction The QCNN is constructed by sequentially combining multiple convolutional and pooling layers, tailored to the number of input qubits. After these layers, a dense layer is applied to the remaining qubits. This dense layer uses an arbitrary unitary gate to perform a final unitary transformation, parameterized by a set of weights, on the quantum state. This final transformation integrates the features extracted by the previous layers and prepares the data for classification or further processing.

Figure 3 illustrates the architecture of the QCNN for binary classification tasks with an input image of size 8×8 pixels. In this setup, the data is encoded into the quantum state using 6 qubits. The figure provides a visual representation of how the quantum convolutional and pooling layers are organized to process the input data and how the dense layer applies a final transformation for classification.

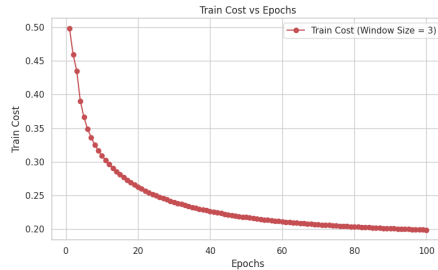
3 Experiments and Results

3.1 Binary Classification

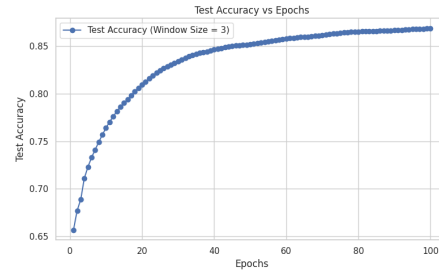
scikit-learn’s Digits Dataset 8x8 In our experiments, we first evaluated the performance of the simulated QCNN using the scikit-learn digits dataset, as referenced in [1]. This dataset consists of 8×8 pixel images of handwritten digits. Our goal was to compare the efficacy of the simulated QCNN against classical CNNs in a binary classification task.

For this evaluation, we implemented a QCNN model using the PennyLane framework, specifically designed to classify images of digits ‘0’ and ‘1’. The QCNN model was trained for 100 epochs with only 80 samples, a relatively small training set compared to typical CNN requirements. Despite the limited data, the QCNN achieved an impressive accuracy of approximately 97%.

In our QCNN model, each of the 64 features in the 8×8 images is encoded into 6 qubits (since $2^6 = 64$) based on the architecture in Figure 3. The convolutional layers of the model are constructed using a combination of U3 and Ising gates. Pooling operations are performed with conditional U3 gates, and the final classification is determined using an arbitrary unitary gate in the dense layer, which outputs the probability of each class.



(a) Training Cost for Digits 8x8 Binary Classification



(b) Testing Accuracy for Digits 8x8 Binary Classification

Fig. 6: Training Cost and Testing Accuracy for scikit-learn’s Digits 8x8 Binary Classification

MNIST Dataset 8x8 This section describes the results we obtained by applying the same model as above on MNIST dataset. To compare the results directly with the referenced model’s dataset, we have downsized the image samples of MNIST dataset from 28×28 to 8×8 and used digits numbered 0 and 1. We have used average pooling to downsample the sample images to 8×8 . The accuracies achieved were around 96% in 100 epochs with the same number of training samples and were comparable to that of the scikit-learn’s digits dataset, solidifying the position that QCNN are capable of generalization with fewer training samples as opposed to its classical counterpart.

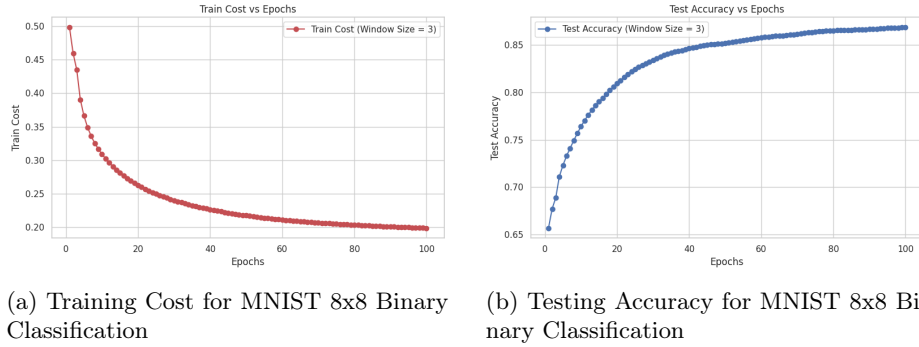


Fig. 7: Training Cost and Testing Accuracy for MNIST 8x8 Binary Classification

CIFAR-10 Dataset grayscale 8x8 This section describes the results on CIFAR-10 dataset downscaled to 8x8 and grayscale to compare the results directly with the above two specifications. We have used average pooling to downsample the image like in MNIST dataset above and normalized them to convert them from colored to grayscale. One thing to consider here is that unlike digits’ dataset and the MNIST dataset, CIFAR-10 does not contain numbers but rather different real world objects like vehicles, animals, and birds. Furthermore downsizing the images from 32×32 and then grayscaleing them leads to a significant loss in information, consequently increasing the complexity of the task. We have used the classes - aeroplane and cars for classification, which are fairly distinguishable just like the digit 0 is from 1. The accuracies achieved were nearly 50% in 100 epochs with the same number of training samples as of the other 2 cases. This result underscores the impact of increasing complexity on the performance of the model.

3.2 Multiclass classification

To further explore the performance of the QCNN, we expanded the classification task to include 10 classes, thereby increasing the complexity of the problem. This

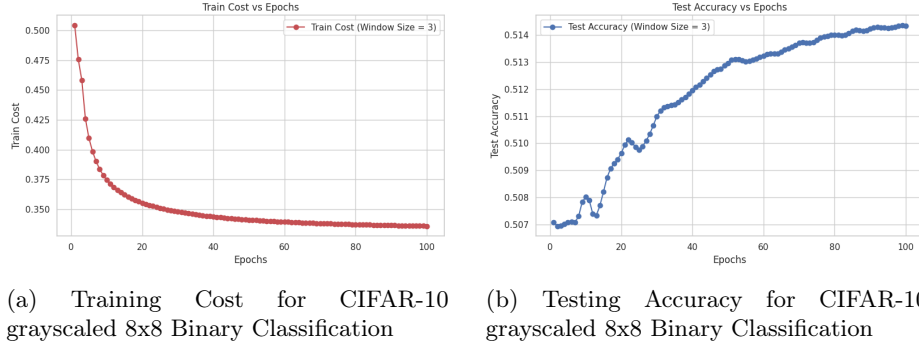


Fig. 8: Training Cost and Testing Accuracy for CIFAR-10 grayscale 8x8 Binary Classification

Table 1: Training and testing accuracy comparison of scikit-learn’s digits dataset and downsized MNIST dataset for different numbers of training samples

No. of Training Samples	Digits Dataset		Downsized MNIST	
	Train Accuracies	Test Accuracies	Train Accuracies	Test Accuracies
2	99.77%	66.14%	99.88%	71.87%
5	91.89%	73.51%	97.52%	81.49%
10	92.42%	81.91%	96.82%	88.10%
20	95.19%	91.35%	97.11%	94.70%
40	95.91%	94.28%	97.06%	95.96%
80	96.80%	96.11%	97.32%	96.57%

Table 2: Training and testing accuracy comparison of scikit-learn’s digits dataset and downsized and greyscaled CIFAR-10 dataset for different numbers of training samples

No. of Training Samples	Digits Dataset		CIFAR-10 Downsized-grayscaled	
	Train Accuracies	Test Accuracies	Train Accuracies	Test Accuracies
2	99.77%	66.14%	98.93%	52.11%
5	91.89%	73.51%	72.33%	49.95%
10	92.42%	81.91%	70.39%	50.84%
20	95.19%	91.35%	62.39%	50.91%
40	95.91%	94.28%	60.15%	51.45%
80	96.80%	96.11%	57.52%	51.87%

section examines the results of the simulated QCNN when tasked with classifying training samples into 10 distinct categories. Our objective is to determine whether the model’s precision remains as high as observed in the binary classification experiments or if it diminishes, as seen in the third experiment. This analysis provides additional insights into the comparative performance of quantum versus classical models.

For classifying images into 10 classes, we require 4 qubits for the arbitrary unitary gate in the dense layer, as $2^3 < 10 < 2^4$. These 4 qubits enable the measurement of probabilities, facilitating the classification process and allowing us to assess how well the QCNN handles a more complex task compared to its performance with fewer classes.

MNIST Dataset 8x8 Building upon our previous findings, we tested the proposed model on downsized MNIST dataset. Since the complexity of the task increased, training samples were increased from just 80 to 4000 which is still a significant amount less than a classical CNN model. The accuracies dropped to just 27% in 100 epochs, which is nowhere near to the performance of CNNs.

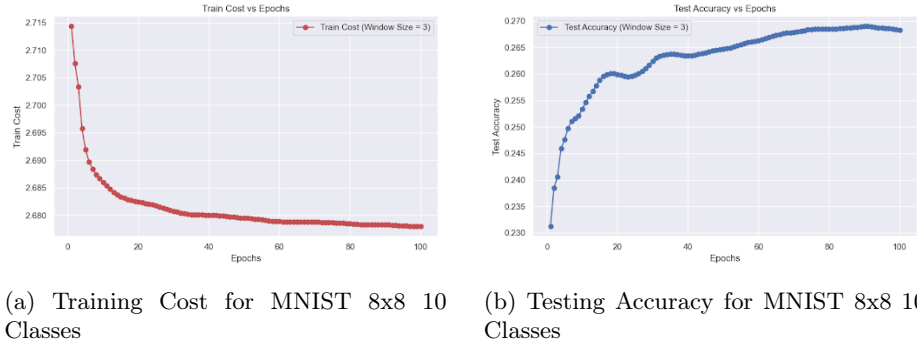


Fig. 9: Training Cost and Testing Accuracy for MNIST 8x8 10 Classes

CIFAR-10 Downsized and grayscaled To strengthen the validity of the previous findings, we tested the model on downsized and grayscaled CIFAR-10 dataset. With 4000 training samples and 100 epochs, the testing accuracy was just above 14%, again a significant drop from the previous results with 27%.

CIFAR-10 Downsized The colored downsized CIFAR-10 dataset had a dimension of $8 \times 8 \times 3$, thus requiring ($2^7 < 192 < 2^8$) 8 qubits to encode each feature in the amplitude of each computational basis as shown in Figure 4. Increasing the number of qubits increased the complexity of the task and thus increased the amount of time required to train the model. With 4000 training

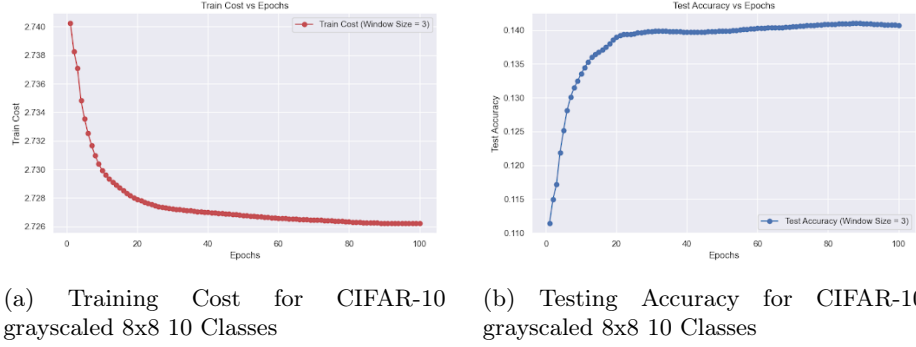


Fig. 10: Training Cost and Testing Accuracy for CIFAR-10 8x8 grayscale 10 Classes

samples and 100 epochs, the testing accuracy was around 11%, again a drop from the previous result.

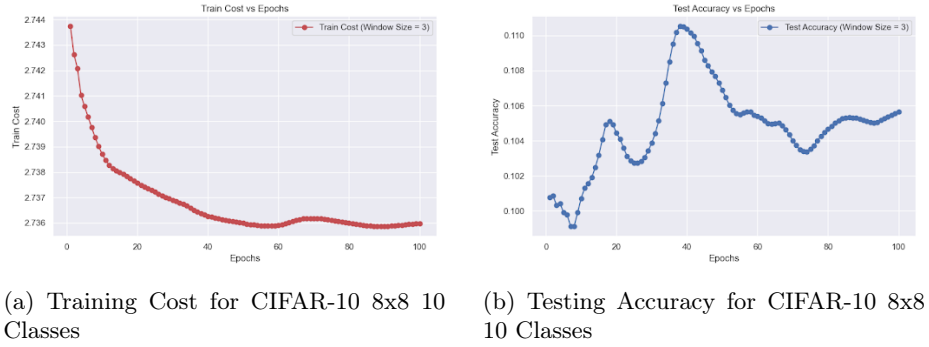


Fig. 11: Training Cost and Testing Accuracy for CIFAR-10 Colored 8x8 10 Classes

3.3 MNIST 28x28

Increasing the complexity of the task further, we tested the model on MNIST dataset. We used 10 qubits to encode the dataset as depicted in Figure 5. With 4000 training samples and 100 epochs, the testing accuracy was around 10%. The time taken to complete the experiment was exponentially greater than all the previous experiments because of the increased size of the dataset as well as the increased number of qubits. With this result, we concluded our experiments.

4 Discussion

This study aimed to evaluate the performance of a simulated QCNN and assess whether it could match the effectiveness of CNNs while utilizing fewer parameters. Building on the results from Caro et al.[1], we applied the QCNN model to various datasets with differing sizes and classification complexities.

Our experiments showed that the QCNN model excelled in simpler tasks, such as binary classification of 8×8 handwritten digit images with just 80 training samples. However, as the complexity of the tasks increased—by shifting from handwritten digits to the more intricate CIFAR-10 dataset and expanding the number of classes from 2 to 10—the model’s performance declined. This indicates a limitation of the simulated QCNN when scaling to more sophisticated applications. The Table 1 and Table 2 shows a comparison of the model’s performance on different datasets and Figure 6 through 11 illustrates the outcomes of various experiments on these datasets, highlighting the performance variations based on the dataset, image size and class count.

Different image size required different number of qubits to encode. An 8×8 image required ($2^6=64$) 6 qubits whereas, a colored CIFAR-10 image of dimensions $8 \times 8 \times 3$ required ($2^7 < 192 < 2^8$) 8 qubits and MNIST dataset with dimensions 28×28 pixels required ($2^9 < 784 < 2^{10}$)10 qubits. As the number of qubits increased, the computational requirements of the model also increased.

The need for increasing qubits led to a significant increase in the amount of computational time, underscoring a major challenge: The execution time can become prohibitively long as the problem size and complexity increments. This is a well known issue in quantum computing, where the addition of more qubits to represent or encode more complex data leads to a dramatic increase in the amount of computational resources required to perform the calculations.

5 Conclusion

Our findings demonstrate that the simulated QCNN model performs well on simpler tasks where both the problem size and complexity are relatively low. The model is both efficient and accurate, showing comparable performance to classical counterparts in terms of data processing capabilities. However, there is still room for improvement when tackling more complex tasks.

Despite the challenges encountered, the potential of QCNNs as a transformative technology is clear. While QCNNs show promise for future advancements in machine learning, realizing their full potential will necessitate further research, development, and innovation.

In summary, QCNNs have the potential to significantly impact the field of machine learning, but achieving their capabilities will require continued exploration and enhancement.

Disclosure of Interest The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Caro, M.C., Huang, H.Y., Cerezo, M., Sharma, K., Sornborger, A., Cincio, L., Coles, P.J.: Generalization in quantum machine learning from few training data. *Nature Communications* **13**(1) (Aug 2022). <https://doi.org/10.1038/s41467-022-32550-3>, <http://dx.doi.org/10.1038/s41467-022-32550-3>
2. Cong, I., Choi, S., Lukin, M.D.: Quantum convolutional neural networks. *Nature Physics* **15**(12), 1273–1278 (Aug 2019). <https://doi.org/10.1038/s41567-019-0648-8>, <http://dx.doi.org/10.1038/s41567-019-0648-8>
3. Khan, M.J., Singh, P.P., Pradhan, B., Alamri, A., Lee, C.W.: Extraction of roads using the archimedes tuning process with the quantum dilated convolutional neural network. *Sensors* **23**(21) (2023). <https://doi.org/10.3390/s23218783>, <https://www.mdpi.com/1424-8220/23/21/8783>
4. Krizhevsky, A., Nair, V., Hinton, G.: Cifar-10 (canadian institute for advanced research) (2009), <http://www.cs.toronto.edu/~kriz/cifar.html>
5. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proceedings of the IEEE* **86**(11), 2278–2324 (1998)
6. Nielsen, M.A., Chuang, I.L.: *Quantum Computation and Quantum Information*. Cambridge University Press (2000)
7. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011)
8. Roh, E.J., Baek, H., Kim, D., Kim, J.: Fast quantum convolutional neural networks for low-complexity object detection in autonomous driving applications. *IEEE Transactions on Mobile Computing* pp. 1–12 (2024). <https://doi.org/10.1109/TMC.2024.3470328>
9. Schuld, M., Sinayskiy, I., Petruccione, F.: Quantum machine learning algorithms: a tutorial review. *Contemporary Physics* **56**(2), 172–185 (2018)
10. Sethna, J.P.: *Statistical Mechanics: Entropy, Order Parameters, and Complexity*. Oxford University Press, Oxford, UK (2006), <https://global.oup.com/academic/product/statistical-mechanics-9780198566779>